LAB EXPERIMENT :

NAME : Richard Benson Raj.B.A

REG NO: 1925260077

COURSE CODE: CSA1711

1.8-Puzzle Problem

```
from collections import deque

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

q = deque([(start, [])])
visited = {start}

while q:
    state, path = q.popleft()
```

```
if state == goal:
    print("Solution:")
    for p in path + [state]:
        print(p[:3])
        print(p[3:6])
        print(p[6:9])
        print()
    break

z = state.index(0)
r, c = divmod(z, 3)

for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
    nr, nc = r + dr, c + dc

    if 0 <= nr < 3 and 0 <= nc < 3:
        nz = nr * 3 + nc
```

```
            new = list(state)
            new[z], new[nz] = new[nz], new[z]
            new = tuple(new)

            if new not in visited:
                visited.add(new)
                q.append((new, path + [state]))
```

output:

Solution:

(1, 2, 3)

(4, 0, 6)

(7, 5, 8)

(1, 2, 3)

(4, 5, 6)

(7, 0, 8)

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

2. 8-Queen Problem

```
def solve(board, row):
    if row == 8:
        for r in board:
            print(" ".join(r))
        return True

    for col in range(8):
        if all(board[i][col] != 'Q' and
               abs(row-i) !=
abs(col-board[i].index('Q'))
```

```
            for i in range(row)):

        board[row][col] = 'Q'

        if solve(board, row + 1):
            return True

        board[row][col] = '.'

    return False


board = [['.' for i in range(8)] for j in
range(8)]
solve(board, 0)
```

output:

. Q . . . . . .

. . . . Q . . .

. . . . Q

. . . . . Q . .

Q . . . . . .

. . Q . . . . .

. . . Q . . . .

. . . . . Q .

3. Water Jug Problem

```
from collections import deque

q = deque([((0, 0), [])])
visited = set()

while q:
```

```
(a, b), path = q.popleft()

if (a, b) in visited:
    continue

visited.add((a, b))

if a == 2:
    for x in path + [(a, b)]:
        print(x)
    break

states = [
    (4, b),
    (a, 3),
    (0, b),
    (a, 0),
```

```
        (a - min(a, 3-b), b + min(a, 3-b)),
        (a + min(b, 4-a), b - min(b, 4-a))
    ]

    for s in states:
        if s not in visited:
            q.append((s, path + [(a, b)]))
```

output:

(0, 0)

(0, 3)

(3, 0)

(3, 3)

(4, 2)

(0, 2)

(2, 0)

4. Crypt-Arithmetic Problem

```
from itertools import permutations

words = "SEND MORE MONEY"

for p in permutations(range(10), 8):
    S,E,N,D,M,O,R,Y = p

    if S == 0 or M == 0:
        continue

    SEND = 1000*S + 100*E + 10*N + D
    MORE = 1000*M + 100*O + 10*R + E
```

```
    MONEY = 10000*M + 1000*O +
100*N + 10*E + Y


    if SEND + MORE == MONEY:
        print("S =", S)
        print("E =", E)
        print("N =", N)
        print("D =", D)
        print("M =", M)
        print("O =", O)
        print("R =", R)
        print("Y =", Y)
        print(SEND, "+", MORE, "=",
MONEY)
      break
```

output:

S = 9

E = 5

N = 6

D = 7

M = 1

O = 0

R = 8

Y = 2

9567 + 1085 = 10652